

Forward and Regressive Algebraic Goal-Oriented Action Planning

Isaac Hsu (Hsu, Chih-Kang)

Introduction

Goal-Oriented Action Planning, or GOAP for short, is an AI technique used in video games that enables AI agents to make intelligent plans to achieve their goals based on their knowledge about the game world, and take actions that change the game state into their desired states. It was introduced to gaming by *F.E.A.R.* and implemented in titles like *Middle-earth: Shadow of Mordor* and *Rise of the Tomb Raider* to create dynamic and adaptive character behavior.

Unlike traditional AI approaches such as state machines and behavior trees that usually require programmers to hardcode the decision-making process, GOAP empowers agents to automate planning at runtime, which lifts the heavy burden off developers and allows agents to improvise solutions to the problems they face. Moreover, the decoupling of actions from goals benefits developers because it is easier to accommodate changes during development and simpler to create a variety of characters with distinct ways of completing the same tasks.

This planning algorithm finds a series of actions with effects that modify the game state step by step to reach a given goal state. Nevertheless, the original GOAP provides only copy operations to alter the game state but does not support arithmetic operations, even for a simple increment. Thus, we can only describe effects that transition between different statuses. However, for RPGs, strategy, and simulation games, it could be more intuitive, flexible, and useful for developers to adopt algebraic expressions for effect formulation such as $HP := HP + \text{Heal}$. This article aims to expand the power and reach of the original algorithm by applying algebra and logic.

The overall procedure of GOAP is to select a goal, formulate a plan for it, and execute the plan. Nonetheless, the article focuses solely on the planning algorithms and does not address goal selection and plan execution. The latter two are more project-specific and straightforward to implement. For instance, we may calculate scores according to custom criteria to choose the best goal among the candidates, and run plans by a system provided by the engine being used or through a specialized mechanism designed for the project.

Related Work

Applying Goal-Oriented Action Planning to Games

In Dr. Jeff Orkin's article "*Applying Goal-Oriented Action Planning to Games*", he introduced a new decision-making solution for game AI so that agents do not just follow predetermined rules to react, but instead reason about solutions on the fly to fulfill their objectives under current circumstances. This technique shifts AI design from predefined behavior modeling to equipping agents with the necessary information and methods to tailor the world to their needs.

More specifically, an AI agent knows some important facts about the game world called **world properties** and stores them in its **world state**. Additionally, it has some desired world states to achieve, which are its **goals**, and some **actions** with **effects** that can alter the world state. Actions may have **preconditions**, such as an NPC needing to be close enough to a door to open it. GOAP is about selecting a goal, finding a sequence of actions (called a **plan**) to change the current state to the goal state, and then executing the plan to actually accomplish the goal in the game world.

Finding a plan for a goal is essentially a search problem in a state space tree, where each edge denotes the action chosen for a branch and each node represents the world state modified by the actions on the path from the root. Just like navigational pathfinding, we can search from a starting point to a goal, or conversely, from a goal to a starting point. In fact, it is usually more efficient to do it backwards because the regressive search space is often smaller.

The original GOAP uses A* as the search algorithm and stores a pair of world states—a current state and a desired state—in each node of the regressive search tree. As the regressive search starts with a goal, the root node stands for the goal and the goal state is copied to the desired state of the root. When newly added properties in the desired state are missing from the current state, their starting values are copied into the current state. To explore adjacent nodes, we need to find the actions with effects that match at least one desired property while differing from the current values. Then we create new nodes for these actions and copy the matching values into the current states and the preconditions into the desired states. To determine if a solution is found, we check whether the current and desired states of the least-cost node are the same. If so, we can form a plan as a sequence of actions by following the path back to the root.

Forward and Regressive Arithmetic Goal-Oriented Action Planning

As previously mentioned, the original regressive GOAP uses pairs of world states to keep track of the search process, but that also introduces a limitation: there should not be conflicts between the preconditions and the effects of an action. For example, if we want an action to switch something on only when it is off, the planner may fail due to the inconsistency between the current and desired values when the desired state is retrodicted (see Table 1).

actions		TargetDead	WpnLoaded	WpnArmed
Attack	preconditions		true	
	effects	true		
LoadWeapon	preconditions		N/A $\sim$ false	true
	effects		true	
DrawWeapon	preconditions			
	effects			true
start		false	false	false
goal		true		
		TargetDead	WpnLoaded	WpnArmed
current		false		
desired		true		
$?\Rightarrow A$		TargetDead	WpnLoaded	WpnArmed
current		true✓	false	
desired		true✓	true	
$?\Rightarrow LW \Rightarrow A$		TargetDead	WpnLoaded	WpnArmed
current		true✓	true X	false
desired		true✓	true $\sim$ false X	true

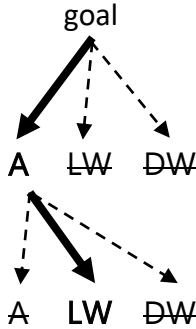


Table 1: This is a modified example from “Applying Goal-Oriented Action Planning to Games”. A seemingly trivial precondition requiring WpnLoaded to be false is added to LoadWeapon, but the original regressive algorithm cannot handle it. However, forward GOAP works well with it.

In my previous article “Forward and Regressive Arithmetic Goal-Oriented Action Planning”, the improved regressive GOAP can address this issue. It removes satisfied world properties from desired states and instead determines search success by checking if the starting state satisfies the desired state of the current least-cost node. In addition, that way we just need to keep the desired states and do not need the current states in the regressive search tree, resulting in lower space demand.

		TargetDead	WpnLoaded	WpnArmed
	start	false	false✓	false
	goal	true		
	$? \Rightarrow A$	TargetDead	WpnLoaded	WpnArmed
	desired	true	true	
	$? \Rightarrow LW \Rightarrow A$	TargetDead	WpnLoaded	WpnArmed
	desired		true → false	true
	$? \Rightarrow DW \Rightarrow LW \Rightarrow A$	TargetDead	WpnLoaded	WpnArmed
	desired		false✓	true

Table 2: This is the same case from Table 1, but the improved regressive GOAP runs properly with the extra precondition of WpnLoaded added to LoadWeapon opposite to its effect.

Furthermore, the original algorithm only uses equality and assignment operations on world properties, so partial satisfaction cannot be described simply through the approach. For instance, an agent needs to repeat an action of collecting resources a few times to make a purchase. This relation of partial satisfaction between the collection and purchase actions cannot be directly formulated. To overcome this limitation, I proposed forward and regressive arithmetic GOAP, which expands the original method and adds the four arithmetic operations on numeric properties for effects and two non-strict order comparators for preconditions and goals: less than or equal to ($\leq$) and greater than or equal to ($\geq$). It also supports range limits on numeric properties, if needed. Moreover, I discussed a few optimization techniques for the regressive search based on lookup tables and a way of making the heuristic admissible for numeric properties.

Forward Algebraic GOAP

In the preceding regressive arithmetic GOAP, an effect applies an arithmetic operation to a numeric property and a constant value, with the result confined to a desired interval so that we can solve the constraints on the property and express them simply as an interval. However, if we want to create an effect that is the sum of two numeric properties, it is not allowed because one operand must be immutable.

To describe general actions and goals, we can use algebraic expressions for effects and algebraic inequalities for goals and preconditions, or even a more general form—

logical formulas combining relational expressions with the basic logical connectives: AND, OR, and NOT. To illustrate, we can formulate healing, a common action in games, as shown below.

$$HP < \text{MaxHP} \wedge SP \geq \text{HealSP} \rightarrow \begin{cases} HP := HP + \text{HealHP} \\ SP := SP - \text{HealSP} \end{cases}$$

The forward algebraic planner is straightforward to implement. The overall procedure is still similar to the original: In each iteration, if the goal is satisfied by the current least-cost node, the plan is built along the path. Otherwise, the node is expanded by finding feasible actions and adding new nodes with their effects to the open set. Likewise, in each node of the search tree, we store the world property values modified by the actions on the path from the root so that we can track the search process and check if the search is complete.

Heuristic Estimation of Relational Expressions and Logical Formulas

The biggest difference from the original GOAP is in the heuristic estimation. Now the constraints of goals and preconditions are relational expressions combined with logical connectives, so a simple way is to evaluate the left and right sides of a relational expression by substituting the values in the current state and to calculate the distance between the two sides. If the relation is satisfied, the distance should be considered zero for admissible heuristics. Otherwise, the distance is the absolute value of the difference between the two sides.

$$d(E_1 \leq E_2) = \begin{cases} 0 & , E_1 \leq E_2 \\ |E_1 - E_2| & , \text{otherwise} \end{cases}$$

where E_i denotes an algebraic expression on one side of an equation or inequality and $\leq$ can be one of the six relational operators except not-equal: $=$, $<$, $\leq$, $>$, or $\geq$.

Regarding an inequation ($\neq$), the distance should also be deemed zero if the relational expression holds true. Otherwise, we can give it a certain positive value as a base relation cost $\mathbb{C}$, since both sides are equal and it is hard to define the distance. For ease of calculation and understanding, we can set $\mathbb{C}$ to one by default.

$$d(E_1 \neq E_2) = \begin{cases} 0 & , E_1 \neq E_2 \\ \mathbb{C} & , \text{otherwise} \end{cases}, \text{ where } \mathbb{C} > 0.$$

So far, this is just half of the answer because in the more general form, we still need a method to calculate the heuristic values for logical formulas composed of relational expressions. Although there are many logical connectives, all logical formulas can be rewritten using only the three basic logical connectives—AND, OR, and NOT—so we will focus solely on them. The operands of logical formulas are entities that can be

evaluated to a truth value, such as Boolean constants, Boolean properties, Boolean operations, and relational expressions.

$$P \equiv \top \mid \perp \mid B \mid R \mid \neg P_1 \mid P_1 \wedge P_2 \mid P_1 \vee P_2 ,$$

where P denotes a logical formula, B denotes a Boolean property, and R denotes a relational expression.

First, we can transform a Boolean property or expression into an equation set to true, as in the examples below.

$$B \equiv B == \top$$

$$B_1 \wedge B_2 \vee B_3 \equiv B_1 \wedge B_2 \vee B_3 == \top$$

Second, concerning Boolean constants, we can map true to 1 and false to 0 in expressions during the heuristic estimation. Moreover, we can reduce truth values in expressions by applying the identity and domination laws, as shown in the following formulas.

$$\top \wedge P \equiv P$$

$$\perp \vee P \equiv P$$

$$\perp \wedge P \equiv \perp$$

$$\top \vee P \equiv \top$$

Simplified logical formulas may end up as single truth values. If a precondition is always true, its constraints can be removed as it is unconditional. If a precondition is always false, the action can be skipped during planning as it is infeasible. If a desired state is always true, then a solution is found. If a desired state is always false, then the search branch is a dead end.

Third, we need to deal with logical NOTs due to their ability to reverse the outcomes of logical formulas and complicate the heuristic estimation. We can remove some of them using the laws of excluded middle, contradiction, and double negation as listed.

$$P \vee \neg P \equiv \top$$

$$P \wedge \neg P \equiv \perp$$

$$\neg(\neg P) \equiv P$$

Similarly, we can convert the negation of a Boolean property or expression into an equation set to false, and take out the logical NOT, as demonstrated below.

$$\neg B \equiv B == \perp$$

$$\neg(B_1 \wedge B_2 \vee B_3) \equiv B_1 \wedge B_2 \vee B_3 == \perp$$

$$\neg(P_1 \wedge P_2) \equiv \neg P_1 \vee \neg P_2$$

$$\neg(P_1 \vee P_2) \equiv \neg P_1 \wedge \neg P_2$$

Furthermore, by applying De Morgan's laws listed above, we can distribute the negations in logical formulas into inner relational expressions. Consequently, we do not have to handle the heuristic estimation for logical NOT, but instead need to support four additional negated relational operators and their distance function:

$$d(E_1 \nsubseteq E_2) = \begin{cases} 0 & , E_1 \nsubseteq E_2 \\ |E_1 - E_2| & , \text{otherwise} \end{cases}, \text{ where } \nsubseteq \text{ can be } \not\leq, \not\geq, \not=, \text{ or } \not\approx.$$

The final step involves merging the heuristic values of relational expressions combined with logical AND/OR. For AND operations, we can sum the distance values of the two relational expressions, since both need to be true for the compound formula to hold. For OR operations, we can take the smaller heuristic value of the two relational expressions, as we should avoid overestimating the heuristic to preserve admissibility.

To explain the reason, imagine a goal with two relational expressions ORed together and an action whose effect satisfies the left expression, so both the left and goal heuristic values become zero after the action. Picture a different scenario with another action that satisfies the right expression, so now the right and goal heuristic values should be zero, regardless of the left heuristic value. Hence, by using the lesser of the two values, we can keep the goal heuristic at zero in both cases. We can formulate the heuristic functions for AND/OR operations on logical formulas and relational expressions, as shown below.

$$h(P_1 \wedge P_2) = h(P_1) + h(P_2)$$

$$h(P_1 \vee P_2) = \min(h(P_1), h(P_2))$$

However, using distance values between the two sides of relational expressions does not guarantee admissible heuristics, as it depends on the operations and the values of the world properties. To maintain admissibility, we can lower the heuristic and/or raise the action costs so that the output of the heuristic functions is always less than or equal to the actual costs. A possible strategy is scaling down the heuristic values of the relational expressions to not exceed the base relation cost, and setting the costs of the actions no less than the base relation cost times the number of relational operations in the goal. This is because the goal state is the only desired state in forward GOAP, and thus it determines the upper bound for the number of comparisons.

In other words, we need to satisfy the following inequalities by multiplying heuristic values by a small enough positive weight w and increasing the action costs:

$$d(R^*) \times w \leq c \wedge \text{cost}(A_i) \geq c \times \#_{\mathcal{R}}(P_G),$$

where R^* represents relational expressions that may be derived in the current search space, c is the base relation cost, A_i is one of the actions, $\#_{\mathcal{R}}$ is a function of the number of relational operations, and P_G is the logical formula of a goal.

Note that heuristics are not weighted when the base relation cost c is returned.

$$h(E_1 \neq E_2) = d(E_1 \neq E_2)$$

$$h(E_1 \preceq E_2) = d(E_1 \preceq E_2) \times w, \text{ where } \preceq \text{ can be } ==, <, \leq, >, \geq, \neq, \neq, \neq, \text{ or } \neq.$$

Heal Example using Forward Algebraic GOAP

To illustrate how the forward algebraic GOAP works, suppose that an agent needs to recover full hit points (HP) using a healing action that restores HP by a base value times a coefficient plus a bonus. Although the coefficient is one and the bonus is zero by default, each can be increased by a buff action once. The planner can solve the problem by healing twice, or by buffing the coefficient and the bonus and then healing once. Additionally, all three actions consume some spell points (SP), and the healing action takes more SP than the two buff actions. Thus, healing twice spends more SP than applying both buffs and healing once.

Heal	preconditions		HP < MaxHP $\wedge$ SP $\geq$ HealSP		
	effects		$\begin{cases} \text{HP} := \text{HP} + \text{Base} \times \text{Coef} + \text{Bonus} \\ \text{SP} := \text{SP} - \text{HealSP} \end{cases}$		
Coef+	preconditions		SP $\geq$ BuffSP $\wedge$ Coef $\leq$ 1		
	effects		$\begin{cases} \text{Coef} := \text{Coef} + \text{CoefInc} \\ \text{SP} := \text{SP} - \text{BuffSP} \end{cases}$		
Bonus+	preconditions		SP $\geq$ BuffSP $\wedge$ Bonus $\leq$ 0		
	effects		$\begin{cases} \text{Bonus} := \text{Bonus} + \text{BonusInc} \\ \text{SP} := \text{SP} - \text{BuffSP} \end{cases}$		
start	MaxHP	HP	SP	HealSP	BuffSP
	100	40	100	40	10
	Base	Coef	CoefInc	Bonus	BonusInc
	30	1	0.5	0	20
goal	HP $\geq$ MaxHP				

Table 3: Actions, starting state, and goal state of the Heal example

If the costs of the actions are the same, the forward planner will find the path of double healing first because it is shorter. It is not necessarily a bad thing if we prefer fewer steps. Alternatively, we can set the action costs to their SP costs so that the search algorithm will find a solution with a lower SP cost instead.

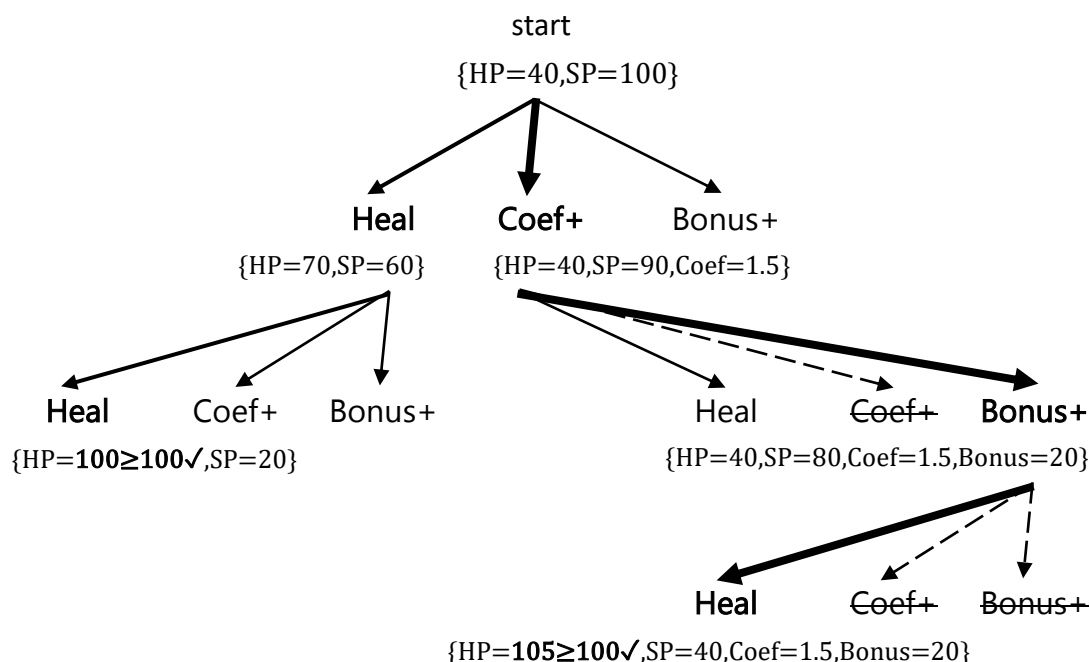


Figure 1: Part of the forward search tree for the Heal example with two possible solutions: Heal⇒Heal and Coef+⇒Bonus+⇒Heal.

Regressive Algebraic GOAP

The general procedure of the regressive algebraic GOAP is similar to the aforementioned improved regressive GOAP, in which we search backwards from a goal, find feasible actions, retrodict previous desired states, and repeat until we discover that the current desired state is satisfied by the starting state.

The regression planner needs to keep track of the desired states during the search process. Since the goals and the preconditions are expressed as logical formulas in the regressive algebraic GOAP, the desired states are as well.

The regressive search starts from a root node with a desired state of the goal constraints. In each iteration, we explore the neighbors of the least-cost node by finding feasible actions with at least one effect on a world property in the desired state. Hence, that effect could produce a previous desired state that might be closer to the starting state and lead to a solution. Moreover, the other effects and the preconditions cannot conflict with the existing constraints in the desired state, or the actions are deemed infeasible.

Assuming that P is the logical formula of a desired state with n world properties, we can write P as an n -ary predicate:

$P \equiv f(x_1, x_2, \dots, x_n)$, where x_i represents a world property.

Then, let A be an action with a precondition formula P_A and an effect E_1 on x_1 :

$A: P_A \rightarrow x_1 := E_1$.

If A is feasible from the desired state of P , we can deduce the constraints of the previous desired state from P using P_A and E_1 as shown below.

$f(E_1, \dots) \wedge P_A$

Therefore, when branching from a search node with a candidate action, we can get the desired state of the new node by substituting the effect expressions and combining with the precondition formula. If there are multiple effects, the effect substitution should be performed in reverse order. Also, if the conjoined logical formula contains contradictory constraints, then the action is infeasible, and the search along the branch can be discontinued. We may identify contradictions by simplifying the expressions, e.g., using the laws of domination and contradiction. On the contrary, we may find a desired formula is always true, which means a solution is found. Nonetheless, because a cheaper path might remain unexplored, we can reset the desired state of the node at this phase, add it to the open set as usual, and allow A^* to find the best solution.

In each iteration of the regressive search, we pick the node with the lowest cost from the open set and substitute the starting values into its desired formula. If the substituted formula is satisfied, then the solution is the sequence of actions leading to the root. Otherwise, continue examining the open set and expanding adjacent nodes with candidate actions until a plan is formulated or the set is empty, at which point the search fails.

To keep the heuristic admissible for the regressive search, we can adapt the idea from the forward algebraic GOAP, which is to reduce the heuristic of the relational expressions to no more than the base relation cost and set the action costs to be no less than the possible heuristic maximum. However, the number of relational operations in the desired formulas may grow as we search further backwards.

Consequently, we use a revised solution: The maximum number of comparisons in a backward step is the number of relational operations in the goal constraints plus the maximum number of relational operations among the actions. This is because evaluating the highest number of relational operations happens when the first step

selects the action whose preconditions have the most comparisons. With the base relation cost c acting as the upper bound for the heuristic of the relational operations, the heuristic maximum of one backward step can be formulated as:

$$c \times \left(\#_{\mathcal{R}}(P_G) + \max_{i=1}^n (\#_{\mathcal{R}}(P_i)) \right),$$

where P_i is the logical formula of an action precondition.

Heal Example using Regressive Algebraic GOAP

To demonstrate how the regressive algebraic search works, we can apply it to the previous Heal example, as shown in the following table.

start	MaxHP	HP	SP	HealSP	BuffSP
	100	40	100	40	10
	Base	Coef	CoefInc	Bonus	BonusInc
	30	1	0.5	0	20
goal	HP $\geq$ MaxHP				X 40 $\geq$ 100
? $\Rightarrow$ Heal	HP+Base $\times$ Coef+Bonus $\geq$ MaxHP				X 70 $\geq$ 100
	$\wedge$ HP < MaxHP				$\checkmark$ 40 < 100
	$\wedge$ SP $\geq$ HealSP				$\checkmark$ 100 $\geq$ 40
Heal $\Rightarrow$ Heal	HP+(Base $\times$ Coef)+Bonus+Base $\times$ Coef+Bonus $\geq$ MaxHP				$\checkmark$ 100 $\geq$ 100
	$\wedge$ HP+(Base $\times$ Coef)+Bonus < MaxHP				$\checkmark$ 70 < 100
	$\wedge$ SP-HealSP $\geq$ HealSP				$\checkmark$ 60 $\geq$ 40
	$\wedge$ HP < MaxHP				$\checkmark$ 40 < 100
? $\Rightarrow$ Bonus+ $\Rightarrow$ Heal	HP+Base $\times$ Coef+Bonus+BonusInc $\geq$ MaxHP				X 90 $\geq$ 100
	$\wedge$ HP < MaxHP				$\checkmark$ 40 < 100
	$\wedge$ SP-BuffSP $\geq$ HealSP				$\checkmark$ 90 $\geq$ 40
	$\wedge$ SP $\geq$ BuffSP				$\checkmark$ 100 $\geq$ 10
Coef+ $\Rightarrow$ Bonus+ $\Rightarrow$ Heal	HP+Base $\times$ (Coef+CoefInc)+Bonus+BonusInc $\geq$ MaxHP				$\checkmark$ 105 $\geq$ 100
	$\wedge$ HP < MaxHP				$\checkmark$ 40 < 100
	$\wedge$ SP-BuffSP-BuffSP $\geq$ HealSP				$\checkmark$ 80 $\geq$ 40
	$\wedge$ SP-BuffSP $\geq$ BuffSP				$\checkmark$ 90 $\geq$ 10
	$\wedge$ Bonus $\leq$ 0				$\checkmark$ 0 $\leq$ 0
	$\wedge$ SP $\geq$ BuffSP				$\checkmark$ 100 $\geq$ 10
	$\wedge$ Coef $\leq$ 1				$\checkmark$ 1 $\leq$ 1

Table 4: Regressive search process for the Heal example with two possible solutions:

Heal $\Rightarrow$ Heal and Coef+ $\Rightarrow$ Bonus+ $\Rightarrow$ Heal. The effect substitution in the desired states is highlighted in grey, and the constraints added from the preconditions are underlined.

Implementation Considerations

Effect Order

The order of effects could be relevant if there is one or more world properties that appear in the previous effects of the same action. Hence, the effects should be applied in the listed order in the forward search. Conversely, in the regressive search, the effects should be applied in reverse order since the process is backward.

For example, consider the following two cases with the same two effects but in different orders. The results are not identical when we substitute a world state $\{x = 1, y = 2, z = 3\}$ into these effects in their listed orders.

$\begin{cases} x := y \\ y := z \end{cases}$	properties	x	y	z
	results	2	3	3

$\begin{cases} y := z \\ x := y \end{cases}$	properties	x	y	z
	results	3	3	3

It can be observed that the right effects in fact imply $x := (y := z)$, while the left ones do not.

Heuristic Adjustment

You may have noticed a small problem in the aforementioned distance function for strict order relational operators ($<$ and $>$). If the values are the same on both sides, the inequalities are still not satisfied, whereas the heuristic values are deemed zero. This is acceptable for the correctness of the algorithm since the heuristic values can be underestimated, and A* always checks whether the goal conditions are met.

Nevertheless, this could make satisfactory and unsatisfactory actions look equally good to A* if both have the same costs, even though one meets the constraint while the other does not. Moreover, if the unsatisfactory action is listed before the other, its node may be explored first, which reduces efficiency. This can apply to their equivalents $\nless$ and $\ngtr$, too. We can fix this issue by adding a small positive offset $\oplus$ to the heuristic values for the strict order relational operations. Note that we may need to adjust heuristic weights or the base relation cost accordingly for the extra offset.

$$h(E_1 \leqslant E_2) = \begin{cases} 0 & , E_1 \leqslant E_2 \\ (|E_1 - E_2| + \oplus) \times w & , \text{otherwise} \end{cases}$$

where $\leqslant$ can be $<$, $>$, $\nless$, or $\ngtr$ and $\oplus$ is an optional small non-negative offset.

In some situations, it may be difficult to predetermine proper heuristic weights w for admissible heuristics. Alternatively, we can first execute the algorithm and keep track of the maximum heuristic values. Then, adjust the weights according to the collected data for admissibility and run again.

Numeric Comparison

Real numbers are mostly represented on computers using floating-point numbers. As they have fixed sizes, their precision is limited. When an arithmetic result cannot fit exactly into the floating-point format, it is rounded to the nearest representable floating-point value by default. Hence, simply comparing floating-point numbers with the equality operator can be unreliable. To illustrate, the below comparison is false for IEEE 754 64-bit double-precision floating-point numbers due to a rounding error: $\text{double}(0.1) + \text{double}(0.2) == \text{double}(0.3)$.

To avoid this problem, we can use rational numbers instead, since they can precisely represent fractions as quotients of two integers. Although we are limited to rational arithmetic, it may be a simple yet robust solution if irrational numbers are not used.

Alternatively, a common technique for floating-point equality is to compare values using an absolute tolerance and a relative tolerance proportional to their magnitudes. If two numbers are not exactly equal but within a small tolerance, they are regarded as equivalent. The relative tolerance is used because floating-point notation is exponent-based, and the absolute tolerance is used because the relative tolerance could be too small to be effective when comparing zero with a very small number close to zero (e.g., subnormal numbers).

That way, however, those relational operators with equality ($==$, $\neq$, $\leq$, $\not\leq$, $\geq$, and $\not\geq$) are actually their approximate versions instead ($\simeq$, $\not\simeq$, $\lesssim$, $\not\lesssim$, $\gtrsim$, and $\not\gtrsim$). Note that trichotomy does not hold with them, so we cannot convert $x \not\leq y$ to $x > y$ like $x \not\leq y \equiv x > y$.

Simplification of Algebraic Expressions and Logical Formulas

Since we are handling algebraic expressions and logical formulas during the search, we can apply techniques and laws in algebra and logic—such as combining like terms and the absorption laws—to simplify the expressions and formulas so that there will be fewer operations in successive iterations for efficiency. We might even notice contradictory constraints in desired states after the simplification or application of certain laws such as domination and contradiction. Conversely, we might prove that some desired constraints always hold, so we could erase them to streamline subsequent steps. On the other hand, even if the expressions and formulas are not rearranged, the results of the algorithm should remain the same, since the simplified versions are logically equivalent to the original ones.

An easy way to further simplify desired constraints is to identify fixed world properties and substitute their starting values into the expressions of the actions and the goal. Fixed world properties are those that do not appear on the left-hand side of assignments in the effects of the actions, so they do not change throughout the planning and can be considered constant. Of course, the substitution applies only to the current planning. The substituted properties should remain in the original action and goal expressions for plan execution and future planning.

To make a compound logical formula of a goal or a desired state simpler to process, we can decompose it into a system of simultaneous logical formulas and rearrange the expressions and formulas into a certain form. That way, we may recognize equivalent constraints more easily and can delete redundant ones. Also, by transposing the constant terms to one side of equations, inequalities, or inequations and the rest to the other side, we may find simultaneous equations or compound inequalities and inequations that can be simplified, such as the following examples.

$$(x == 1 - 3y) \wedge (x + 3y == 2) \equiv (x + 3y == 1) \wedge (x + 3y == 2) \equiv \perp$$

$$(x \neq 1 - 3y) \vee (x + 3y \neq 2) \equiv (x + 3y \neq 1) \vee (x + 3y \neq 2) \equiv \top$$

$$(x \neq 1 - 3y) \wedge (x + 3y > 2) \equiv (x + 3y \neq 1) \wedge (x + 3y > 2) \equiv x + 3y > 2$$

Extension

In the proposed method, the operations in the algebraic expressions can include mathematical functions, not just the four basic arithmetic operations. As an instance, in the Heal example, we can cap the healing amount by setting it to the minimum of the current and maximum HP after the first effect of the Heal action.

$$HP < \text{MaxHP} \wedge SP \geq \text{HealSP} \rightarrow \begin{cases} HP := HP + \text{Base} \times \text{Coef} + \text{Bonus} \\ HP := \min(HP, \text{MaxHP}) \\ SP := SP - \text{HealSP} \end{cases}$$

We can further extend this algorithm to work on mathematical objects other than scalars such as complex numbers, vectors, matrices, and even sets, provided that there is a way to measure the distance between them for the heuristic estimation. For instance, we may calculate the Manhattan or Euclidean distance for the heuristic when comparing pairs of complex numbers, vectors, and matrices. The heuristic weights for admissibility may be difficult to predetermine since there are many more numbers involved than the scalar case, but we can still employ the hindsight trick mentioned earlier. Nonetheless, for mathematical objects whose terms cannot be safely transposed in an equation, such as sets, we usually have to take an alternative route.

Set Theory Extension

Just as with the previous heuristic formulas for numbers, whenever a relation between two set expressions is established, the heuristic value is considered zero. Regarding the distance between two unequal set expressions on the sides of an equation, we can calculate the size of their symmetric difference because the members on only one side make the difference:

$$h(S_1 == S_2) = \begin{cases} 0 & , S_1 == S_2 \\ |S_1 \triangle S_2| \times w & , \text{otherwise} \end{cases}$$

where S_i denotes a set expression on one side of a set theory relation and w is an optional positive heuristic weight (default: 1) used for admissibility or efficiency.

With respect to heuristic estimation for set inclusion, only additional members in the left set matter, whereas extra elements in the right set are irrelevant. Thus, we can measure the size of the left set minus the right set for the heuristic.

$$h(S_1 \subseteq S_2) = \begin{cases} 0 & , S_1 \subseteq S_2 \\ |S_1 \setminus S_2| \times w & , \text{otherwise} \end{cases}$$

Likewise, we can use a similar formula for proper inclusion. Nonetheless, because the right set is required to contain at least one more member not in the left set, we also need to check the size of the right set minus the left set. If the size is zero, we can increase the distance by one so that the effects of adding different members to the right side are preferable. Additionally, no set can strictly contain a universal set U , so the base relation cost c is returned in that case.

$$h(S_1 \subsetneq S_2) = \begin{cases} 0 & , S_1 \subsetneq S_2 \\ c & , S_1 == U \\ |S_1 \setminus S_2| \times w & , S_1 \not\subseteq S_2 \wedge S_1 \neq U \wedge |S_2 \setminus S_1| > 0 \\ (|S_1 \setminus S_2| + 1) \times w & , S_1 \not\subseteq S_2 \wedge S_1 \neq U \wedge |S_2 \setminus S_1| \leq 0 \end{cases}$$

Considering inequation and non-inclusion relations, it is difficult to determine their heuristics since operations that unbalance the sides may immediately satisfy the unmet negated relations. For example, a different member is added to the left or a shared member is removed from the right. Hence, even if an unsatisfied set inequation has more shared members between both sides than another, we cannot definitively say that its distance is shorter or longer. We can simply use the base relation cost as in the earlier heuristic formula for numeric inequations.

$$h(S_1 \neq S_2) = \begin{cases} 0 & , S_1 \neq S_2 \\ c & , \text{otherwise} \end{cases}$$

$$h(S_1 \not\subseteq S_2) = \begin{cases} 0 & , S_1 \not\subseteq S_2 \\ c & , \text{otherwise} \end{cases}$$

where $\not\subseteq$ can be the negation of proper or improper inclusion.

To process set complements, if the universal set size is manageable, we may convert them to set differences: $S^C = \mathbf{U} \setminus S$. Alternatively, we can use the following formulas for common set operations involving complements.

$$\begin{aligned}
S_1^C \cap S_2 &= S_2 \setminus S_1 \\
S_1^C \cap S_2^C &= (S_1 \cup S_2)^C \\
S_1^C \cup S_2 &= (S_1 \setminus S_2)^C \\
S_1^C \cup S_2^C &= (S_1 \cap S_2)^C \\
S_1^C \setminus S_2 &= (S_1 \cup S_2)^C \\
S_1 \setminus S_2^C &= S_1 \cap S_2 \\
S_1^C \setminus S_2^C &= S_2 \setminus S_1 \\
S_1^C \triangle S_2 &= (S_1 \triangle S_2)^C \\
S_1^C \triangle S_2^C &= S_1 \triangle S_2
\end{aligned}$$

To calculate the complement size for heuristic estimation, we can apply the formula: $|S^C| = |\mathbf{U}| - |S|$. However, if the universal set is infinite, the complement size may be infinite, too. In that case, we may cap the heuristic at the base relation cost to keep A* running. In practice, we are more likely to deal with finite sets due to limitations in data types, container implementation, operating systems, or hardware.

For set membership heuristics, provided there is a distance function $\mathcal{D}$ between the members (e.g., the absolute difference for a set of numbers), we can use the distance from the element to the closest member in the set. That way, we may guide the search toward more promising paths. Nevertheless, if no suitable distance functions are available or the given set is empty, the base relation cost can be used instead.

$$h(m \in S) = \begin{cases} 0 & , m \in S \\ \min_{x \in S} (\mathcal{D}(x, m)) \times \mathbb{w} & , m \notin S \wedge (S \neq \emptyset \wedge \exists \mathcal{D}) \\ \mathbb{c} & , m \notin S \wedge (S == \emptyset \vee \nexists \mathcal{D}) \end{cases}$$

where m denotes an element of the universal set or an expression evaluating to such an element. $\mathbb{w}$ may need to be adjusted according to $\mathcal{D}$ for admissible heuristics.

Concerning non-membership operations, the heuristic should similarly be zero if the relation holds. Otherwise, we can just return the base relation cost.

$$h(m \notin S) = \begin{cases} 0 & , m \notin S \\ \mathbb{c} & , \text{otherwise} \end{cases}$$

To handle complements for membership heuristics, we can convert them to the original sets via the equations below and then apply the preceding two functions.

$$\begin{aligned}
m \in S^C &= m \notin S \\
m \notin S^C &= m \in S
\end{aligned}$$

As you may have noticed, the ranges of distance values depend on the input set sizes and could easily exceed the default base relation cost of 1. To make the heuristic admissible, we can scale it down by the ratio of the base relation cost to the universal set size ($c/|U|$) or a smaller positive weight. This is because the maximal set size is the largest distance between two set expressions built from the basic set operations: intersection, union, difference, symmetric difference, and complement.

Nonetheless, the Cartesian product creates sets of ordered pairs, and the power set produces sets of subsets, significantly expanding the scope of the universal set. Consequently, to compute the maximal potential set for admissible heuristics, we need to identify the action and goal expressions involving the two operations, especially those nested in the effects. Next, we apply the following formulas for the sizes of the Cartesian product and the power set, respectively.

$$|S_1 \times S_2| = |S_1| \cdot |S_2|$$

$$|\mathcal{P}(S)| = 2^{|S|}$$

Since property and expression values are mutable, we use their universal sets instead when applying the preceding two formulas to derive the maximal potential set size.

For instance, the maximal set size for an expression with a Cartesian product after a double power set can be calculated as follows:

$$\text{MaxSetSize}(\mathcal{P}(\mathcal{P}(S_1)) \times S_2) = |\mathcal{P}(\mathcal{P}(U_1)) \times U_2| = 2^{(2^{|U_1|})} \cdot |U_2|,$$

where U_1 and U_2 denote the original universal sets of S_1 and S_2 , respectively.

Furthermore, unlike goal constraints, the precondition and effect expressions of an action may be used repeatedly to deduce the desired constraints in the regressive search. Hence, the maximal potential set can grow larger than in previous steps. Therefore, we must consider potentially retrodicted expressions and may need to repeat the computation multiple times (up to the maximum search depth) so that we obtain the maximal set size used to scale down the heuristic for admissibility.

Conclusion

GOAP is a powerful planning architecture for AI agents to generate a series of actions to fulfill their goals. Thus, we do not need to write explicit code for different cases. Instead, we give agents the required information and means, enabling them to find their own solutions. Moreover, it allows AI to predict future situations based on previous predictions, which is not something traditional game AI methods—including state machines, behavior trees, and utility AI—can easily achieve. It also makes the agents smarter and the gaming experience more surprising and engaging.

However, the original algorithm provides only equality comparisons for goals and preconditions and copy operations for effects. Consequently, its expressiveness is limited, and many conditions and actions involving numeric operations cannot be directly described and processed.

With the proposed approach, we can greatly expand the supported operations to include arithmetic operators, mathematical functions, relational operators, and logical connectives. This allows us to compose algebraic expressions and logical formulas for complex goals, preconditions, and effects. In addition, we can extend the domain from scalars to collections of numbers like vectors and matrices, or even to mathematical objects not composed of numbers, such as non-numerical sets, if needed. Through algebra and logic, we can formulate digital conditions and actions in mathematical language, empowering AI agents to find steps toward their goals on their own.

References

Orkin, J. (2003) *Applying Goal-Oriented Action Planning to Games*

Hsu, I. (2024) *Forward and Regressive Arithmetic Goal-Oriented Action Planning*

Links of Example Code

The C++ code for the examples mentioned is available at the links below.

- Weapon example
<https://github.com/Isatin/GOAPPrototypes/blob/main/GOAP/WeaponExample/WeaponExample.cpp>
- Heal example
<https://github.com/Isatin/GOAPPrototypes/blob/main/AlgebGOAP/HealExample/HealExample.cpp>